

Especificación del Flujo de Arranque, Gestión de Periféricos y Ejecución

Consola de Videojuegos Retro sobre FPGA

Proyecto PBL — Curso Diseño Digital 1

Checkpoint 1: Flowchart y especificación de registros CSR

Contents

1	Introducción y arquitectura de referencia	2
1.1	Convención de grupos	2
1.2	Leyenda de formas	2
2	Fase 1 — Arranque y diagnóstico	3
2.1	Diagrama	3
2.2	Descripción	3
3	Fase 2 — Gestión de periféricos	5
3.1	Diagrama	5
3.2	Descripción	5
4	Fase 3 — Navegación y selección local	6
4.1	Diagrama	6
4.2	Descripción	7
5	Fase 4 — Ejecución, audio y seguridad en caliente	8
5.1	Diagrama	8
5.2	Descripción	8
6	Puntos pendientes y observaciones de revisión	9

1 Introducción y arquitectura de referencia

Este documento especifica formalmente el flujo de control del firmware que gobierna la consola de videojuegos retro implementada sobre el SoC descrito en `diagrama_soc_digital1_v3.svg`. El procesador (`femtorv32_quark`, RV32I) se trata como caja negra; toda la lógica aquí descrita corresponde al firmware en C que controla, a través de registros mapeados en memoria (CSR), los periféricos diseñados por cada equipo del curso.

1.1 Convención de grupos

Cada nodo del flujo indica, entre paréntesis, el o los grupos responsables del módulo RTL y/o firmware involucrado. Esta trazabilidad es la que permite que, aunque la plataforma sea única y compartida, cada equipo pueda verificar de forma aislada la porción del flujo que le corresponde.

Grupo	Módulo RTL / responsabilidad	Rango de memoria
A	<code>bram.v</code> — memoria de arranque / firmware	0x000000 – 0x3FFFFFFF
B	<code>perip_uart.v</code> — diagnóstico serie	0x400000 – 0x40FFFFF
C	<code>perip_spiram.v</code> — RAM de trabajo externa	0x410000 – 0x41FFFFF
D	<code>perip_spiflash.v</code> — almacenamiento de assets/firmware	0x420000 – 0x42FFFFF
E	<code>perip_ps2kbd.v</code> — teclado PS/2	0x430000 – 0x43FFFFF
F	<code>perip_ps2mouse.v</code> — mouse PS/2	0x440000 – 0x44FFFFF
G	<code>perip_nes.v</code> — control tipo NES	0x450000 – 0x45FFFFF
I	<code>perip_i2s.v</code> — audio	0x470000 – 0x47FFFFF
J	<code>perip_display.v</code> — framebuffer / panel	0x480000 – 0x4FFFFFFF
K	<code>software_juegos</code> — lógica de juego / firmware de integración	—
L	LEDs RGB de puerto y pantallas laterales (periférico adicional)	—

Table 1: Trazabilidad grupo → módulo, según el mapa de memoria del SoC.

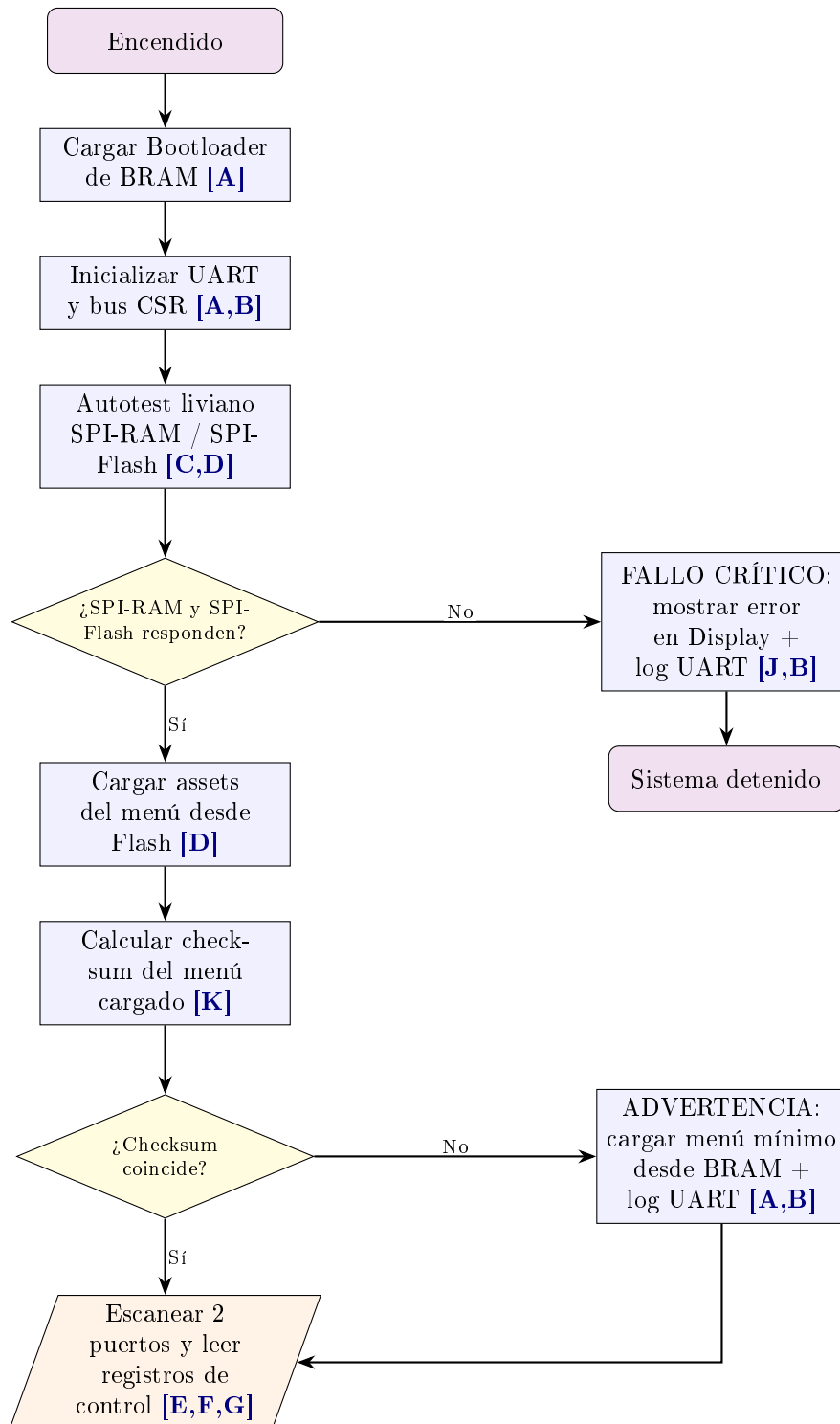
Nota: el grupo H (I2C / EEPROM de puntajes) no participa del flujo principal de arranque—menú—ejecución; su interacción se documenta por separado en los subdiagramas de protocolo I²C (§6).

1.2 Leyenda de formas



2 Fase 1 — Arranque y diagnóstico

2.1 Diagrama



2.2 Descripción

La Fase 1 garantiza que el sistema pueda alcanzar un estado usable incluso ante fallos parciales de hardware, distinguiendo dos niveles de severidad:

- **Fallo duro** (SPI-RAM o SPI-Flash no responden al patrón de prueba): detiene el sistema, pero mantiene el reporte de error operativo porque BRAM, UART y Display están en rangos

de memoria independientes de SPI-RAM/Flash (véase Tabla 1), por lo que no dependen del subsistema que falló.

- **Fallo blando** (checksum del menú no coincide): el sistema no se detiene; degrada a un menú de emergencia embebido en BRAM y continúa el flujo normal hacia el escaneo de puertos.

Algorithm 1 Fase 1 — Arranque y diagnóstico

```

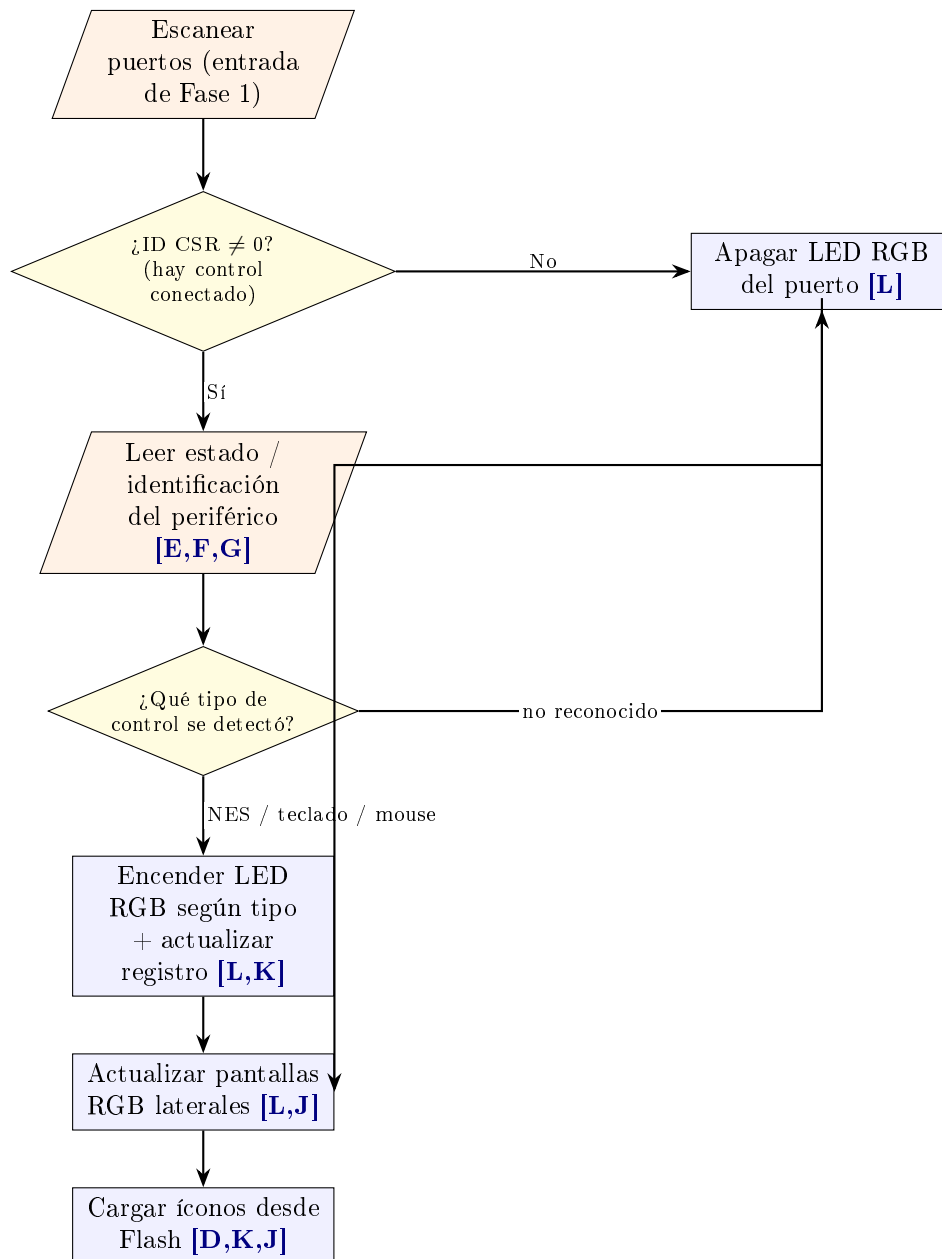
1: cargar_bootloader_bram()
2: inicializar_uart_y_bus_csr()
3: ok ← autotest_liviano(SPIRAM, SPIFLASH)
4: if ¬ ok then
5:   mostrar_error_critico(Display, UART)
6:   detener sistema
7: else
8:   cargar_assets_menu_desde_flash()
9:   chk ← calcular_checksum(menu_cargado)
10:  if chk ≠ chk_esperado then
11:    advertencia_no_bloqueante(BRAM, UART)
12:  end if
13:  escanear_puertos()
14: end if

```

▷ continúa en Fase 2

3 Fase 2 — Gestión de periféricos

3.1 Diagrama



3.2 Descripción

Cada uno de los dos puertos físicos se escanea leyendo su registro CSR de identificación. Un valor de ID = 0 indica puerto vacío. Si hay un dispositivo, se lee su firma real (para PS/2, el comando estándar 0xF2 “**read device ID**”, soportado nativamente por teclados y mouses PS/2) para distinguir el tipo de periférico conectado. El resultado —reconocido o no— retroalimenta tanto el indicador visual (LED de puerto, pantallas RGB) como el registro de estado que consulta la lógica de juego (Grupo K).

Algorithm 2 Fase 2 — Gestión de periféricos (por cada puerto)

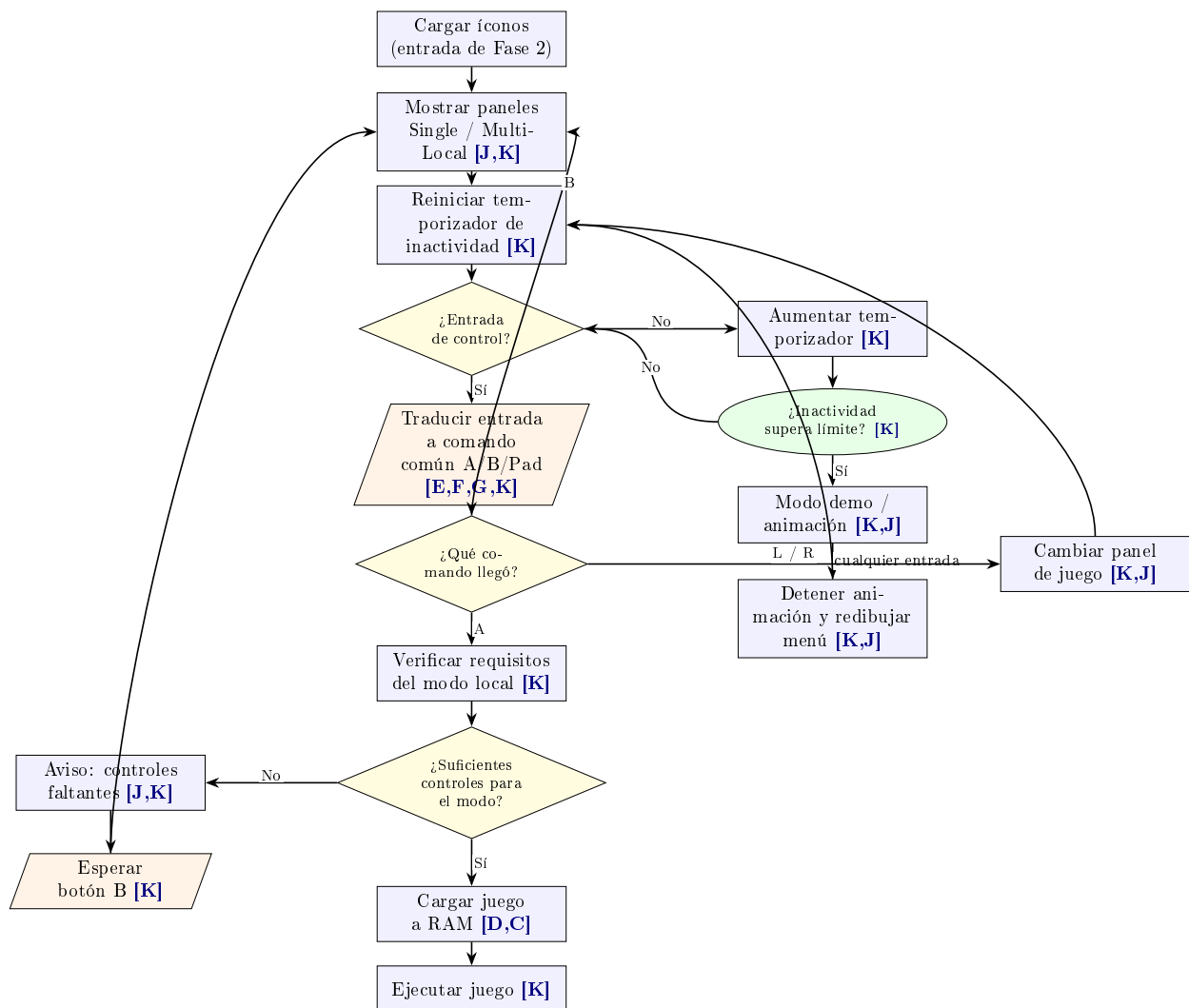
```

1: for all puerto  $\in \{\text{PS2, NES}\}$  do
2:   if leer_id_csr(puerto)  $\neq 0$  then
3:     firma  $\leftarrow$  leer_firma(puerto)
4:     if firma  $\in \{\text{teclado, mouse, NES}\}$  then
5:       encender_led(puerto, firma); actualizar_registro_K(puerto, firma)
6:     else
7:       apagar_led(puerto)
8:     end if
9:   else
10:    apagar_led(puerto)
11:   end if
12: end for
13: actualizar_pantallas_rgb_laterales()
14: cargar_iconos_desde_flash()
    
```

▷ continúa en Fase 3

4 Fase 3 — Navegación y selección local

4.1 Diagrama



4.2 Descripción

Esta fase implementa un menú con *screensaver* por inactividad (temporizador reiniciado ante cualquier entrada; al superar el umbral se entra en modo demo, del cual se sale con cualquier entrada nueva) y la navegación estándar de una consola: L/R cambia de panel, A valida e intenta cargar el modo seleccionado, B regresa al menú. La validación de requisitos (**MultiCheck**) evita cargar un modo que necesite más controles de los detectados en la Fase 2.

Algorithm 3 Fase 3 — Navegación y selección local

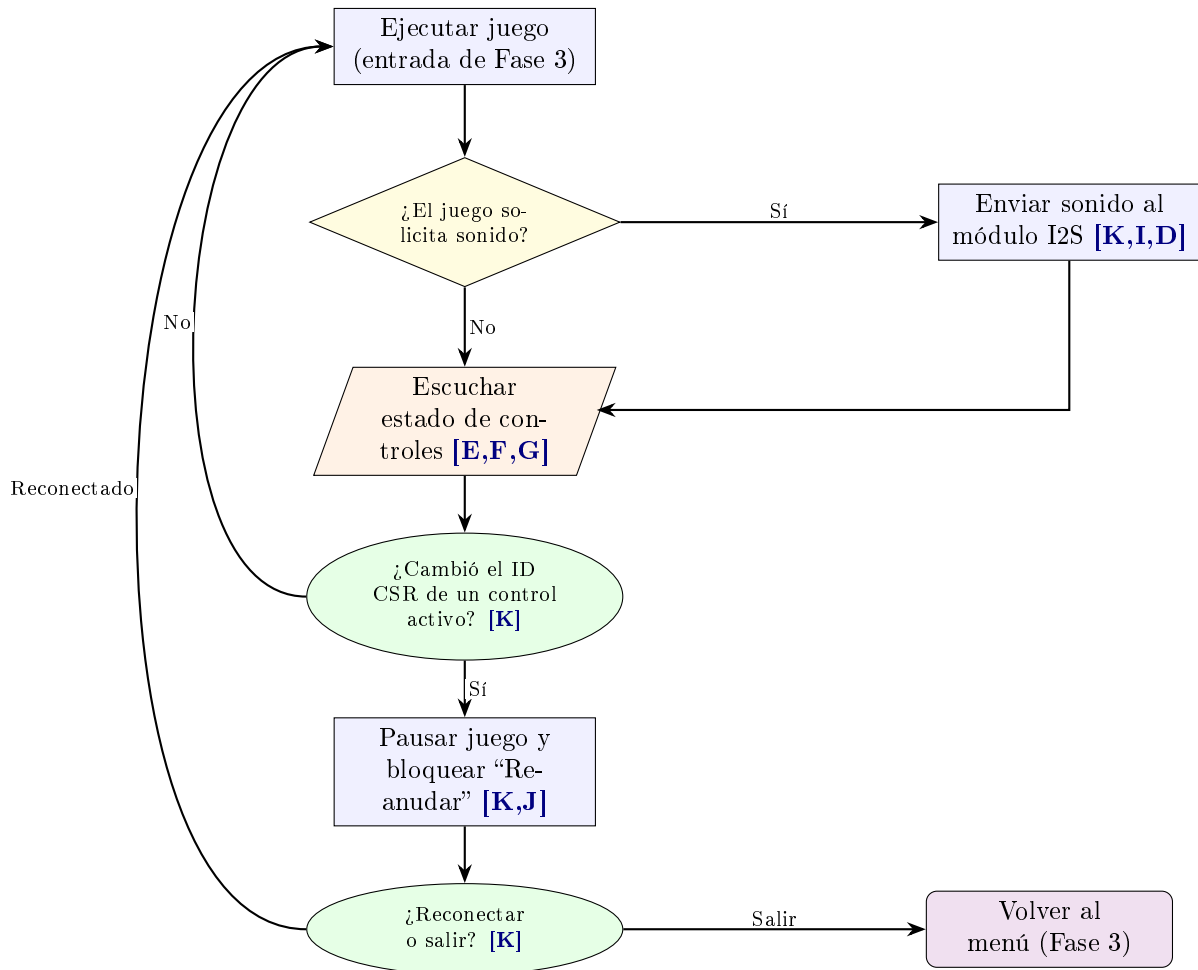
```

1: mostrar_menu(); timer_idle ← 0
2: loop
3:   if hay_entrada_de_control() then
4:     timer_idle ← 0
5:     cmd ← traducir_a_comando_comun()
6:     if cmd = L or cmd = R then
7:       cambiar_panel(); continue
8:     else if cmd = A then
9:       if suficientes_controles(modos_actuales) then
10:        cargar_juego_a_ram(); ejecutar_juego()           ▷ Fase 4
11:      else
12:        avisar_controles_faltantes(); esperar_boton_B(); mostrar_menu()
13:      end if
14:    else if cmd = B then
15:      mostrar_menu()
16:    end if
17:  else
18:    timer_idle ← timer_idle + 1
19:    if timer_idle > límite then
20:      modo_demo()           ▷ hasta nueva entrada
21:    end if
22:  end if
23: end loop

```

5 Fase 4 — Ejecución, audio y seguridad en caliente

5.1 Diagrama



5.2 Descripción

Durante la ejecución, el firmware atiende solicitudes de audio del juego (enviadas al módulo I2S) y, en cada iteración, vigila el registro CSR de identificación de los periféricos activos. Si el ID de un control que estaba en uso cambia –lo cual indica una desconexión o intercambio en caliente– el juego se pausa y se bloquea la opción de reanudar hasta que el mismo tipo de control vuelva a conectarse o el usuario decida salir al menú. Este mecanismo depende directamente de que el mapa de memoria exponga un registro de identificación estático y legible por polling en cada periférico de entrada (Grupos E, F, G).

Algorithm 4 Fase 4 — Ejecución, audio y hot-plugging

```

1: while jugando do
2:   if juego_solicita_sonido() then enviar_a_i2s()
3:   end if
4:   escuchar_estado_controles()
5:   if id_csr_cambio(control_activo) then
6:     pausar_juego(); bloquear_reanudar()
7:     repeat
8:       esperar_evento()
9:     until reconectado or usuario_pide_salir
10:    if usuario_pide_salir then
11:      volver a Fase 3 (mostrar_menu)
12:    end if
13:  end if
14: end while

```

6 Puntos pendientes y observaciones de revisión

El flujo es lógicamente completo y coherente con la arquitectura de bus CSR del SoC (ningún nodo queda sin salida, todos los ciclos cierran correctamente y los dos niveles de fallo –duro y blando– están bien diferenciados). Se dejan documentados, sin embargo, los siguientes puntos abiertos detectados en la revisión, que deben resolverse antes de pasar a la etapa de RTL/ASM:

1. **Detección del control NES (Grupo G).** A diferencia de PS/2, el protocolo NES no expone un comando de identificación estándar. Debe definirse explícitamente en la especificación de `perip_nes.v` si la detección se resuelve mediante (a) un adaptador que emula una firma PS/2 falsa, o (b) un pin dedicado de presencia física, con confirmación manual desde el menú.
2. **Correspondencia “2 puertos” vs. 3 periféricos CSR.** El flujo escanea 2 puertos físicos (uno PS/2 combinado + uno NES), mientras que el mapa de memoria expone 3 registros CSR independientes (teclado, mouse, NES). Debe documentarse explícitamente en la especificación de registros cómo el puerto PS/2 físico único se traduce a los dos rangos CSR de teclado y mouse.
3. **Umbral del temporizador de inactividad.** El flowchart no fija un valor numérico; debe incorporarse a la tabla de constantes del firmware (`software_juegos`, Grupo K).
4. **Módulos con subdiagrama aún no desarrollado.** Las páginas de flujo específicas para I2S, F-Mouse y Flash dentro del archivo fuente están a nivel de esbozo y deben completarse por los equipos responsables antes del checkpoint de RTL simulado.
5. **Interacción I2C / EEPROM (Grupo H).** No forma parte del flujo principal aquí documentado (arranque–menú–ejecución); se dispara asincrónicamente al finalizar una partida (guardado de puntajes) y debe especificarse en un documento de flujo independiente, ya esbozado en las páginas “H.1” y “H.2” del archivo original.